

Set-up and registration

International Marketing Analytics



Dr Arne Floh, Senior Lecturer in Marketing
WU Vienna, Institute for International Business

arne.floh@wu.ac.at

[Office hours: book online \(Google Calendar\)](#)

Today: from zero to a working set-up

Accounts

Tools

Repo

Project

Check

AI

What you need



- Your laptop and charger
- Windows 10/11 or a recent macOS
- About 10 GB free disk space
- Admin rights to install software

Have at hand



- Your WU e-mail and its password
- Your phone, for two-factor sign-in
- Your student ID or enrolment confirmation

How we work



- One step at a time, all together
- Check your neighbour's screen
- Stuck? Raise your hand, keep going with the next step



Goal: by the end of today the check script prints “ready” on every laptop.

Register your accounts



Part 1 of 7 · GitHub, GitHub Education,
DataCamp

Three registrations

Accounts

Tools

Repo

Project

Check

AI



GitHub

Stores code and its history;
hosts the course repository

github.com/signup



GitHub Education

Free student benefits,
including GitHub Copilot

education.github.com/pack



DataCamp

Online Python courses; four
certificates count 20 %

Invitation link on Canvas



Same e-mail everywhere: your WU student address unlocks the free plans.

Accounts

Tools

Repo

Project

Check

AI

1 Open github.com/signup

2 **WU e-mail:** sign up with your student address

3 **Username:** short and professional, e.g. anna-berger

4 **Verify** the code GitHub sends to your inbox

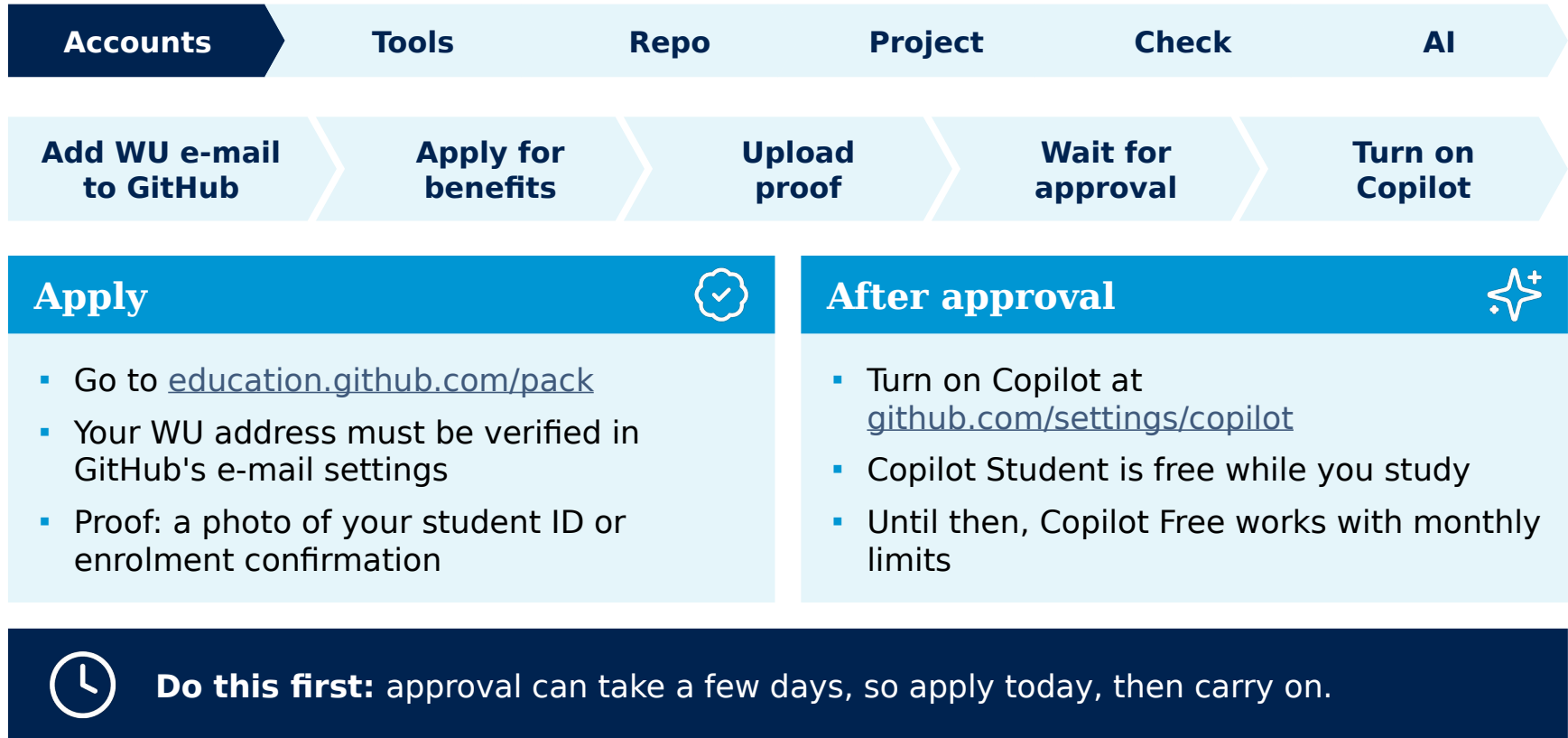
5 **Two-factor sign-in:** GitHub Mobile or an authenticator app

Why it matters



- Your username appears on every commit you make
- A clean GitHub profile is part of your CV
- Optional: share your group's case study in a repository
- The free plan is all you need

GitHub Education and Copilot



Accounts

Tools

Repo

Project

Check

AI

- 1 **Open** the invitation link on Canvas
- 2 **Sign up** with your WU e-mail, or log in
- 3 **Join** the classroom International Marketing Analytics
- 4 **Find** the four assigned courses under Assignments
- 5 **Upload** each certificate to Canvas by its deadline

6
months free premium access

4
courses with certificates

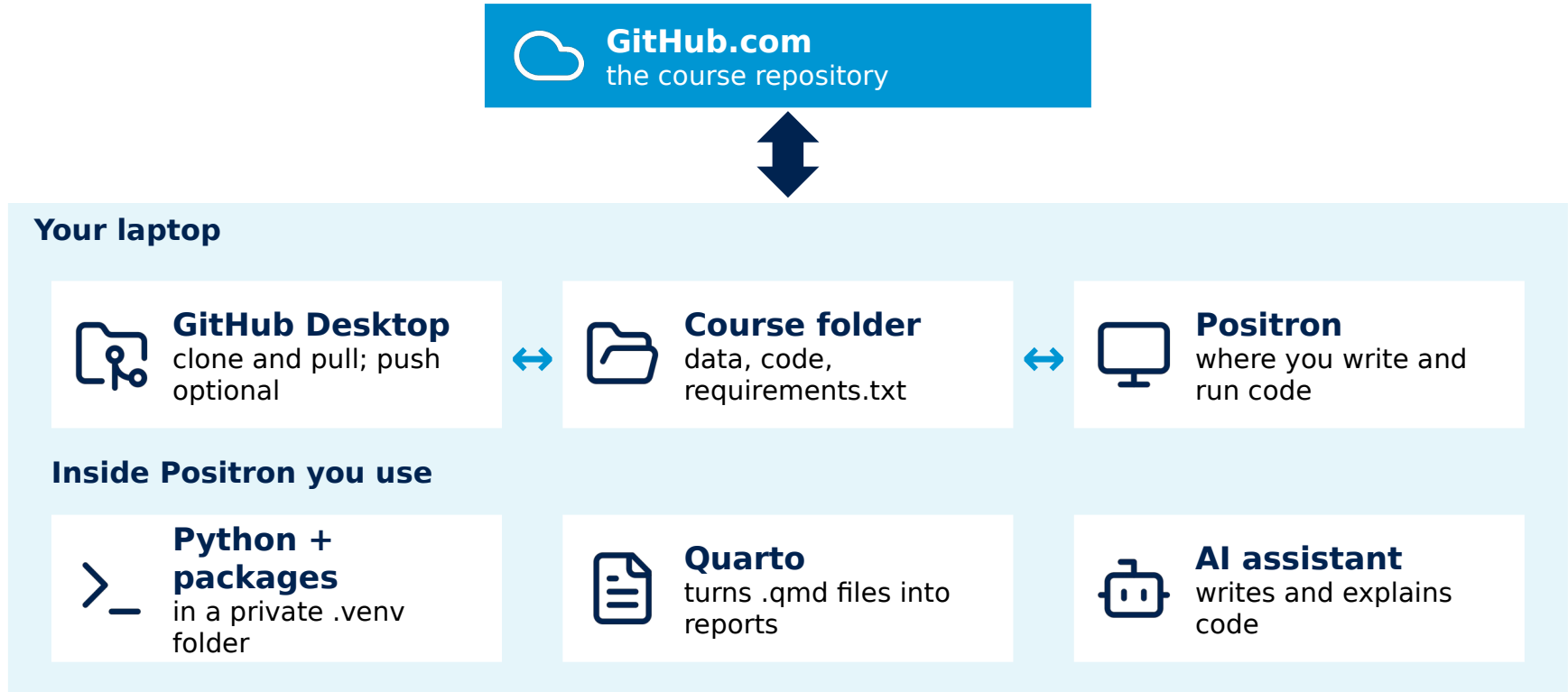
20 %
of the final grade

Install the tools



Part 2 of 7 · Python, Positron, Quarto, GitHub
Desktop, the course repository

How the pieces fit together



Step 2: install Python 3.14.8

Accounts

Tools

Repo

Project

Check

AI

Windows



- **Download:** Python install manager
- Install it with a double-click
- Start menu → Terminal: `py install 3.14`

```
py -3.14 --version
```

macOS



- **Download:** macOS installer (Python 3.14.8)
- Open the .pkg and click through
- Apple silicon and Intel

```
python3.14 --version
```



Both downloads: python.org/downloads

Or let Positron install it: Command Palette → Python: Install Python via uv

Step 2: install Positron, Quarto and GitHub Desktop

Accounts

Tools

Repo

Project

Check

AI

Positron



- The editor for code, data and charts
- Version 2026.09, free
- Welcome page checks that Python is ready

positron.posit.co/download

Quarto



- Turns text and code into reports and slides
- Check in a terminal:
quarto --version

quarto.org/docs/get-started

GitHub Desktop



- Clone and pull with clicks; push if your group shares a repository
- Installs Git for you
- Sign in with your GitHub account

desktop.github.com



Restart Positron after installing Quarto, so it finds it.

Step 3: get the course repository

Accounts

Tools

Repo

Project

Check

AI

**Copy the URL
(below)**

**GitHub Desktop:
Clone repository**

**Pick a folder
without spaces**

**Positron:
Open Folder**

Course repository (public)

[https://github.com/arnefloh-wu/
international-marketing-analytics-teaching](https://github.com/arnefloh-wu/international-marketing-analytics-teaching)

Folder: e.g. C:\Users\anna\wu\ima or ~/wu/ima.
Avoid spaces, umlauts and OneDrive or iCloud
folders.

Clone: your own copy of a repository; a public
one needs no sign-in

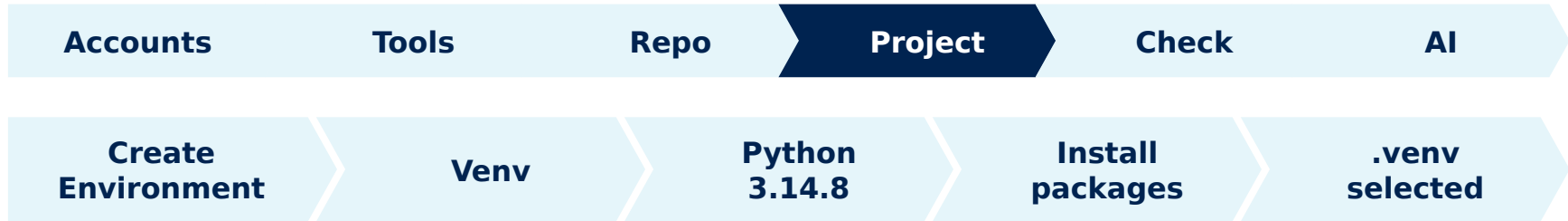
Pull: get the latest materials; do it before every
session

Commit and push: optional: share work in a
group repository



In Positron: File → Open Folder. The cloned folder is now your project.

Step 4: set up the project's Python



Positron then



- Start: Command Palette → Python: Create Environment
- Creates .venv inside the project
- Installs all course packages
- Starts Python from .venv

Interpreter picker
top right

Python 3.14.8 (.venv) ▼

Packages missing? Positron console

```
%pip install -r requirements.txt
```



Once per project: Positron remembers the .venv and starts it whenever you open the folder.

Step 5: run the check

Accounts

Tools

Repo

Project

Check

AI

Positron console (Windows and macOS)

```
%run setup/check_setup.py
```

Example output (shortened)

```
Packages
[ok] polars          1.44.2    data wrangling
[ok] plotnine        0.15.8    charts
[ok] great_tables    1.0.0     tables
[ok] statsmodels      0.15.0    regression
...
ready
```

Reading it



- [ok]: in place
- [!!!]: the line says what to fix
- Ends with “ready”: done



Next: the full test with test_stack.qmd (next slide).

Step 5: test the whole stack

test_stack.qmd · cells 1 and 2

```
# 1 Install: run once with Ctrl/Cmd + Enter
%pip install polars plotnine great-tables \
    statsmodels pyarrow jupyter
# 2 Load
import polars as pl
import statsmodels.formula.api as smf
from great_tables import GT
from plotnine import ggplot, aes, geom_line
```

cells 3 and 4

```
# 3-4 polars and great_tables
sales = pl.read_csv(URL, try_parse_dates=True)
summary = (sales
    .filter(pl.col("week").dt.year() == 2025)
    .group_by("country")
    .agg(pl.col("revenue_eur_k").sum()))
GT(summary)
```

cell 5

```
# 5 plotnine: weekly sales, AT and DE
two = sales.filter(
    pl.col("country").is_in(["AT", "DE"]))
ggplot(two, aes("week", "sales_units_k",
    color="country")) + geom_line()
```

cell 6

```
# 6 statsmodels: regression for Austria
at = sales.filter(
    pl.col("country") == "AT").to_pandas()
model = smf.ols("sales_units_k ~ price_eur"
    " + spend_tv_k + spend_paid_search_k",
    data=at).fit()
```

Result: a table, a chart, the coefficients



Run it: cell 1 once with Ctrl/Cmd + Enter, then Preview (Ctrl/Cmd + Shift + K).

Step 6: connect your AI assistant

Accounts

Tools

Repo

Project

Check

AI

- 1 Command Palette:** Configure Language Model Providers
- 2 Choose GitHub Copilot** and sign in with GitHub
- 3 Open the chat:** View: Show Posit Assistant

Providers

GitHub Copilot: chat and completions; free with Copilot Student or Copilot Free

Posit AI Pass: chat and completions; about USD 20 a month

Anthropic API key: chat; pay as you go



Our rule: the AI writes, you check. Run the code, read the output and keep a short prompt log for each submission.

How Positron works



Part 3 of 7 · panes, running code, projects, shortcuts, Posit Assistant, Claude

New ▼ Open ▼

case-study ▼

Python 3.14.8 (.venv) ▼

1 Explorer
files of the open folder

data/
analysis.py
report.qmd
requirements.txt

2 Editor
write .py and .qmd files

%%
sales = pl.read_csv(...)
sales.head()

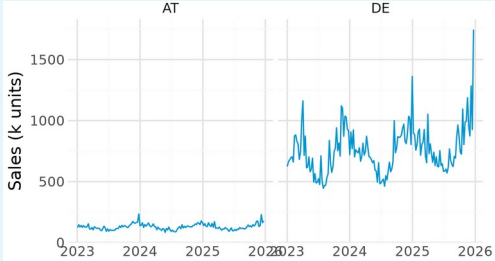
3 Console and Terminal
the Panel: code runs, results print

>>> sales.height
936

4 Variables
Session pane: what is in memory

sales 936 × 18
summary 6 × 3
model OLS results

5 Plots
Session pane: your charts



From code to result

**Write code
in the editor**

**Ctrl/Cmd
+ Enter**

**It runs in
the console**

**Objects in
Variables**

**Charts in
Plots**

Run code



- Ctrl/Cmd + Enter: the selection or current statement
- Ctrl/Cmd + Shift + Enter: the whole file, or the current cell
- # %% starts a cell in a .py file

Look at results



- Variables: every object, its type and size
- Data Explorer: click a table, or type %view sales
- Plots: earlier charts, zoom, save as PNG



Console or terminal? The console speaks Python. The terminal speaks to your computer: pip, git and quarto commands go there.

Projects in Positron: one folder for everything

A project is a folder

```
case-study/
├── .venv/           private Python + packages
├── .git/            version history
├── .gitignore       what Git leaves out
├── data/            raw data, never edit by hand
├── analysis.py       code, split into # %% cells
├── report.qmd        text, code and results
└── requirements.txt  packages and versions
```

Open the folder, not single files: File
→ Open Folder

Positron remembers the interpreter
and open files per folder

Relative paths: data/sales.csv works
on every laptop

Switch projects with the project
switcher, top right



.venv stays on your laptop: .gitignore keeps it out of GitHub. requirements.txt travels instead, so your group rebuilds the same environment with one command.

Start a new project from a template

New Folder
from Template

Python
Project

Name and
location

Python 3.14
+ venv

Create

Positron sets up



- The folder with a .venv inside
- Git: .git and a .gitignore
- A running Python session
- An empty file to start with

Then, in the Positron console

```
%pip install polars plotnine great-tables
```

```
%pip freeze > requirements.txt
```

Line 1 installs packages, line 2 writes their versions to requirements.txt.



Where: New ▼ (top left), the project switcher, or Command Palette → Workspaces: New Folder from Template.

Shortcuts worth knowing

Action	Windows	macOS
Run selection or current statement	Ctrl + Enter	Cmd + Enter
Run the file or current cell	Ctrl + Shift + Enter	Cmd + Shift + Enter
Insert a code cell (# %%)	Ctrl + Shift + I	Cmd + Shift + I
Restart the Python session	Ctrl + Shift + 0	Cmd + Shift + 0
Clear the console	Ctrl + L	Ctrl + L
Preview a Quarto document	Ctrl + Shift + K	Cmd + Shift + K
Command Palette	Ctrl + Shift + P	Cmd + Shift + P
Help for the word under the cursor	F1	F1
Accept an AI suggestion	Tab	Tab

Posit Assistant: the AI inside Positron

What it sees 	What it does 	How you steer it 
▪ Your open files and project ▪ Loaded data, variables and plots ▪ Console history	▪ Explains, writes and fixes code ▪ Runs code after you approve ▪ Fix and Explain on any error	▪ <code>/plan</code>: a plan before any code ▪ <code>@file</code>: point it to a file ▪ Permission mode: keep Normal

Open it

View: Show Posit Assistant

Command Palette, or the Assistant icon in the Activity Bar

Memory: an AGENTS.md file in your project tells it your conventions, for example “use polars and plotnine”.

Completions: ghost text



- Grey suggestions appear as you type
- Tab accepts, Ctrl/Cmd + → word by word
- Esc or keep typing to dismiss

Chat: Posit Assistant



- Ask in plain English, one step at a time
- Check every result it reports
- Export the chat: your prompt log

Good prompts

"Sum revenue by country for 2025 with polars and sort it."
"Explain this error in two sentences, then fix it."



Privacy: prompts, variable names and types go to the model provider; rows of data only if you ask. Never paste personal data.

Claude in Positron: two ways

1 Claude in Posit Assistant

- Command Palette → Authentication: Configure Language Model Providers → Anthropic
- Paste an API key from the Claude Console
- Pick a Claude model in the chat

2 Claude Code, the agent

- Extensions view: search Claude Code (Anthropic), Install
- Or run claude in the Positron terminal
- Sign in with a paid Claude plan or Console account

Costs: API keys are paid per use; Claude Pro or Max does not include API access.



Optional: the course default is GitHub Copilot, free with GitHub Education. Never share your API key.

Working with Claude Code in your project

**Open your
project**

**Start
Claude**

**Ask in plain
English**

**Approve
changes**

**Run and
check**

Positron terminal: Windows, macOS, then start

```
irm https://claude.ai/install.ps1 | iex  
curl -fsSL https://claude.ai/install.sh | bash  
claude
```

CLAUDE.md: standing instructions

use polars, plotnine, great_tables and
statsmodels only; explain every model
in two sentences

Good prompts

"Read test_stack.qmd and explain each cell in one sentence."
"Add a residual plot to my regression; do not change the data."



Same rules as for any AI: you own every number, keep a prompt log, and never paste personal data.

Work with GitHub Desktop



Part 4 of 7 · the window, the everyday workflow, working as a group

GitHub Desktop at a glance

1 Current repository
ima-teaching ▼

2 Current branch
main ▼

3 Fetch origin
↻ **now**

4 **Changes (2)** History

analysis.py

report.qmd

Add Austria sales chart

Commit to main

5 summary, then commit

6 **report.qmd**

```
at = sales.filter(...)  
- at.head()  
+ ggplot(at, aes("week",  
+   "sales_units_k"))  
+   + geom_line()
```

1 **Current repository**
which project you work on

2 **Current branch**
main is all we need

3 **Fetch / Pull / Push origin**
sync with GitHub

4 **Changes**
files edited since last commit

5 **Summary + Commit**
save a snapshot with a message

6 **Diff**
red removed, green added

Course repository: you only pull. Work in your own folder; a group repository is optional.

PAGE 28

INTERNATIONAL MARKETING ANALYTICS · WT 2026/27

EFMD
EQUIS
ACCREDITED

AACSB
ACCREDITED

ASSOCIATION
AMBA
ACCREDITED

The everyday workflow

**Fetch, then
Pull origin**

**Copy files into
your own folder**

**Work in
Positron**

**Optional:
commit, push**

Course repository



- Required: pull before every session
- Read-only for you: copy files into your own folder before editing

Group repository



- Optional: one member publishes a private repository
- Pull, work, commit, push
- One person per file where you can

Good commits



- If you commit: small steps, often
- Summary says what changed: "Add AT sales chart"
- Push when you stop



Merge conflict in a group repository? Open the file, keep the lines you want, delete the markers (<<< === >>>), then commit and push.

Working on the case study together

**Pull course
repository**

**Copy into
your folder**

**Positron:
.venv**

**Combine:
2 ways**

**Upload
to Canvas**

Project folder



- Copy data/case, report.qmd and requirements.txt
- report.qmd next to the data folder
- Create Environment from requirements.txt

Way 1: simple



- One member keeps report.qmd
- The others send code chunks and text
- Render the combined file every week

Way 2: GitHub



- Optional: Publish repository, private
- Invite on github.com: Collaborators
- Pull, commit, push



Only Canvas counts: upload report.qmd and report.html; your commit history is not assessed.

Write reports with Quarto



Part 5 of 7 · how it works, the YAML header,
code cells, output formats

Quarto: text, code and results in one file



report.qmd

```
---
title: "Alpenglow: sales in Austria"
author: "Anna Berger"
format: html
---

## Weekly sales

```{python}
at = sales.filter(pl.col("country") == "AT")
at.head()
```

Austria is our smallest market.
```

1 YAML header: settings between the two --- lines

2 Markdown: headings (##), **bold**, lists, links

3 Code cells: ```{python} ... ``` run Python; the output lands under the cell



Preview: Ctrl/Cmd + Shift + K in Positron. Render: quarto render report.qmd

The YAML header: settings for the whole document

Top of report.qmd

```
---
title: "Alpenglow: sales in Austria"
author: "Anna Berger"
date: today
format:
  html:
    toc: true
    code-fold: true
execute:
  warning: false
---
```

| | |
|------------------------|--|
| title, author | shown at the top |
| date: today | the day you render |
| format: html | the output; its options indented below |
| toc: true | table of contents |
| code-fold: true | code hidden behind a "Code" button |
| warning: false | no warnings in the output |



Indentation matters: two spaces per level, no tabs, and a space after every colon.

Code cells: options and numbers in the text

Inside report.qmd

```
```{python}
#| label: fig-sales
#| fig-cap: "Weekly sales in Austria"
#| echo: false
ggplot(at, aes("week", "sales_units_k")) + geom_line()
```
```

The chart covers `{python} at.height` weeks (@fig-sales).

#| label: fig-sales
names the cell; fig- makes it a figure

#| fig-cap:
the caption under the chart

#| echo: false
show the result, hide the code

`{python} ...`
a computed number in the text

@fig-sales
"Figure 1", with a link

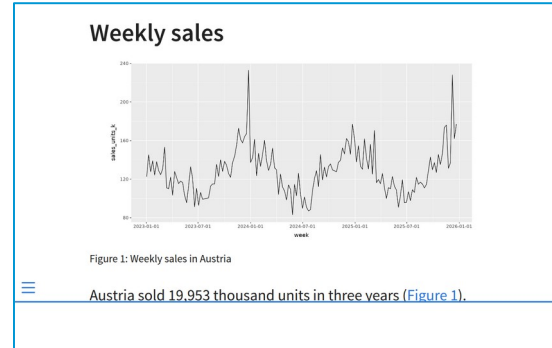


Why it matters: when the data change, you render again and every chart, table and number in the text updates. No copy and paste into Word.

One source, many output formats



html
web page (default)



revealjs
slides in the browser



typst
PDF, no LaTeX
needed



docx
Word, to edit further

YAML: several formats

```
format:
  html: default
  revealjs: default
  typst: default
  docx: default
```

Terminal: all formats, or one

```
quarto render report.qmd
quarto render report.qmd --to docx
```

Also: pptx, dashboard, pdf (LaTeX)

Python packages for this course



Part 6 of 7 · install and load; polars, plotnine,
Great Tables, statsmodels

Install packages: three ways

1 Windows and macOS: Positron console (easiest)

```
%pip install polars plotnine great-tables
```

2 Windows: Positron terminal

```
.venv\Scripts\python -m pip install -r requirements.txt # all course packages  
.venv\Scripts\python -m pip install polars # one package
```

3 macOS: Positron terminal

```
.venv/bin/python -m pip install -r requirements.txt # all course packages  
.venv/bin/python -m pip install polars # one package
```



Install name and import name can differ: pip install great-tables → import great_tables; scikit-learn → sklearn; pymc-marketing → pymc_marketing.

The Packages pane: install without code

**Packages icon
(Activity Bar)**

**Install
Package**

**Search name,
pick version**

**Restart session
when asked**

PACKAGES Python 3.14.8 (.venv)

| | | |
|--------------|--------|------------|
| great-tables | 1.0.0 | |
| pandas | 3.0.6 | ● attached |
| plotnine | 0.15.8 | |
| polars | 1.44.2 | ● attached |
| statsmodels | 0.15.0 | ● attached |
| ... | | |

What else it does



- Shows which packages are loaded (attached)
- Update one, or Update All Packages
- Flags known security issues
- Keeps versions in line with requirements.txt
- Offers to install a missing package after an error

Load packages: import at the top of every file

**Install once
per laptop**

**Import in every
file and session**

**Use the short
name: `pl.read_csv()`**

Windows and macOS: the same code

```
import polars as pl
import statsmodels.formula.api as smf
from plotnine import ggplot, aes, geom_line
from great_tables import GT

print(pl.__version__)    # 1.44.2
```

Conventions

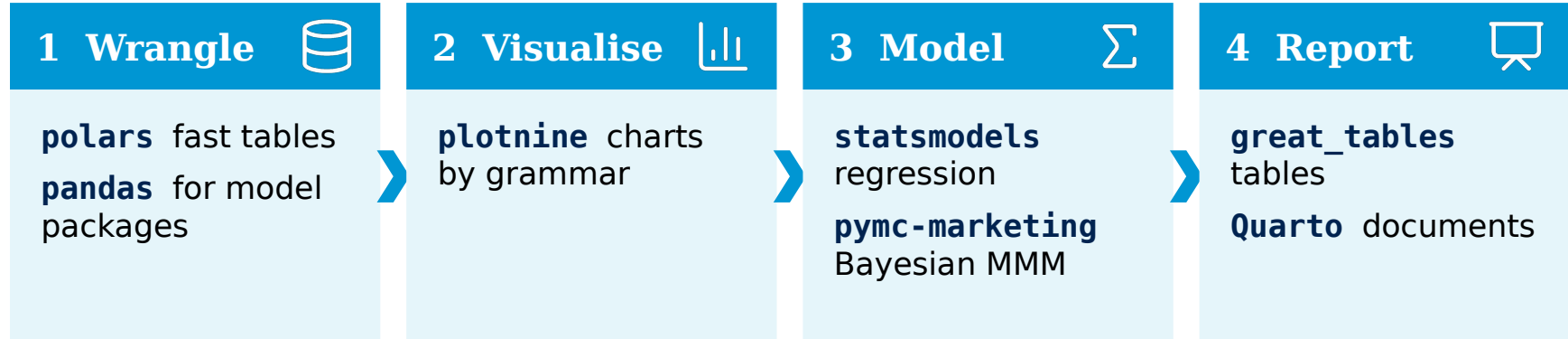


- `pl` and `smf` are the usual short names
- Imports go at the top
- Import only what you use



ModuleNotFoundError? Either the wrong interpreter is selected (pick the one in `.venv`) or the package is missing: `%pip install it`.

Your toolkit, from data to report



22

packages pinned in requirements.txt

1

command installs them all

0

euros: all open source



Online you will also meet pandas and matplotlib: older but common; the ideas carry over.

Revenue and TV spend per country, 2025

```
import polars as pl

sales = pl.read_csv("data/mmm/alpenglow_weekly.csv",
                    try_parse_dates=True)

summary = (
    sales
    .filter(pl.col("week").dt.year() == 2025)
    .group_by("country")
    .agg(pl.col("revenue_eur_k").sum().alias("revenue"),
         pl.col("spend_tv_k").sum().alias("tv"))
    .sort("revenue", descending=True)
)
```

DataFrame: a table with typed columns

Expressions: `pl.col("tv").sum()`
says what to compute

Chaining: one step per line, read top to bottom

Fast: multi-threaded, written in Rust

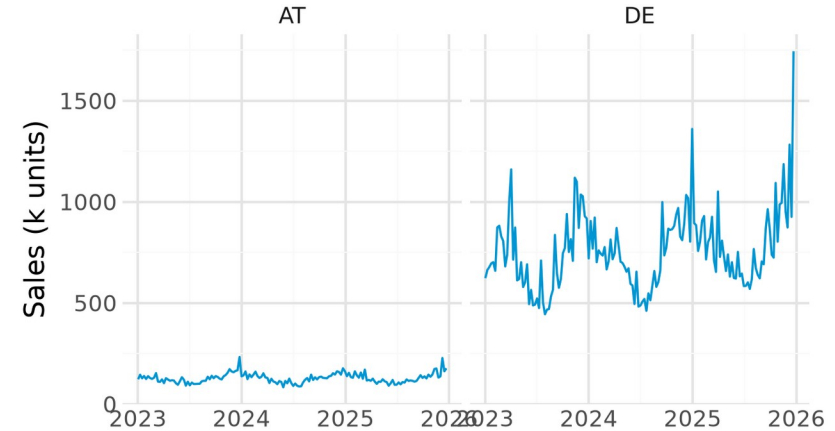


Result: six rows, one per country. Germany leads with about EUR 131 million revenue in 2025. Need pandas? Add `.to_pandas()`.

plotnine: charts with the grammar of graphics

Weekly sales, Austria and Germany

```
(ggplot(sales.filter(
  pl.col("country").is_in(["AT", "DE"])),
  aes(x="week", y="sales_units_k"))
+ geom_line(color="#0096D3")
+ facet_wrap("country")
+ scale_x_date(date_breaks="1 year",
  date_labels="%Y")
+ labs(x="", y="Sales (k units)")
+ theme_minimal())
```



The output of the code on the left

ggplot()
the data

aes()
columns to x, y

geom_
the marks

facet_
one panel per
group

theme_
the look

Data + mapping + marks + panels + theme: each + adds one layer.

Great Tables: tables for reports

The polars summary as a table

```
from great_tables import GT

(GT(summary)
 .tab_header(title="Revenue and TV spend, 2025")
 .fmt_number(columns=["revenue", "tv"], decimals=0)
 .cols_label(country="Country",
              revenue="Revenue (k EUR)",
              tv="TV spend (k EUR)"))
```

Header and labels: readable names instead of column codes

Number formats: thousands separators, currency, percent

Works with polars and pandas; renders in Quarto reports

| Revenue and TV spend, 2025 | | |
|----------------------------|-----------------|------------------|
| Country | Revenue (k EUR) | TV spend (k EUR) |
| DE | 130,650 | 5,434 |
| FR | 66,306 | 2,363 |
| IT | 47,950 | 2,478 |
| NL | 33,234 | 812 |
| PL | 23,112 | 616 |
| AT | 21,984 | 695 |

Weekly sales in Austria, 2023 to 2025

```
import statsmodels.formula.api as smf

at = sales.filter(pl.col("country") == "AT").to_pandas()
model = smf.ols(
    "sales_units_k ~ price_eur + spend_tv_k"
    " + spend_paid_search_k",
    data=at).fit()
print(model.summary())
```

| Coefficient | Estimate |
|--------------------------|----------|
| Intercept | 250.63 |
| price_eur | -48.31 |
| spend_tv_k | 0.41 |
| spend_paid_search_k | 4.03 |
| R ² (n = 156) | 0.43 |



Reading it: EUR 1,000 more paid search in a week goes with about 4,000 more units sold, holding price and TV constant.

Later in the course: scikit-learn for logistic regression and pymc-marketing for Bayesian MMM.

Update within Positron



Part 7 of 7 · Positron, Python, the
project's .venv, packages

Keep your set-up up to date



Positron itself

Updates itself; restart to apply
By hand: Help → Check for Updates



Python

Python: Install Python via uv
Command Palette; pick 3.14



The project's .venv

Tied to the Python that made it
New Python? Rebuild the .venv (next slide)



Packages

After a pull: `%pip install -r requirements.txt`
One package: `%pip install --upgrade polars`



Course materials: not in Positron. Pull in GitHub Desktop before every session.

Move a project to Python 3.14

- 1 **Install 3.14:** Command Palette → Python: Install Python via uv → 3.14
- 2 **Stop Python:** power icon in the Console
- 3 **Delete .venv:** Explorer, right-click the .venv folder → Delete
- 4 **Create Environment:** Venv → Python 3.14.8, tick requirements.txt
- 5 **Check:** pick Python 3.14.8 (.venv), then %run setup/check_setup.py

Interpreter picker

Python 3.13.5 (.venv)

before

Python 3.14.8 (.venv) ✓

after

The check then reports **3.14.8** and ends with **ready**.



Why rebuild? a .venv cannot be upgraded; the old Python can stay installed.

When something goes wrong

| Problem | Fix |
|--|--|
| Windows: python or py is not recognised | Reinstall the Python install manager, then run py install 3.14 |
| No .venv in the Interpreter picker | Open the course folder, not a single file; then pick Python 3.14.8 (.venv) |
| The check shows Python 3.13 or older | Move the project to Python 3.14 (Part 7) |
| ModuleNotFoundError | Select the .venv interpreter, then %pip install the package |
| macOS: externally-managed-environment | You used the system Python: select the project's .venv first |
| Quarto preview does not start | Restart Positron; run quarto check in the terminal |
| Clone fails in GitHub Desktop | Sign in again under Accounts in the settings; check the URL |
| Copilot is not offered | GitHub Education still pending: use Copilot Free meanwhile |

Checklist: before you leave today

Accounts



- ☐ GitHub account with two-factor sign-in
- ☐ GitHub Education application sent
- ☐ DataCamp classroom joined

Laptop



- ☐ Python 3.14, Positron, Quarto, GitHub Desktop
- ☐ Course repository cloned and open
- ☐ .venv created and selected
- ☐ “ready” and the test report renders
- ☐ Posit Assistant connected



Not ready yet? Post the failing step in the Q&A forum on Canvas before Session 2.



VIENNA UNIVERSITY OF
ECONOMICS AND BUSINESS

**INSTITUTE FOR INTERNATIONAL
BUSINESS**

Welthandelsplatz 1, 1020 Vienna,
Austria

DR ARNE FLOH

Senior Lecturer in Marketing

F +43-31336-6367

arne.floh@wu.ac.at

www.wu.ac.at/en/welthandel/arne-floh

[Office hours: book online](#)